Hand-Gesture-Driven Smart Home Control System

Kevin Wang
University of Washington
+1 (408) 203-9122
kwang522@uw.edu

Jenelle Ng
University of Washington
+1 (206) 201-4760
Jenl24@uw.edu

Dalton Brockett
University of Washington
+1 (360) 984-1665
daltonbo@uw.edu

Bella Zhang
University of Washington
+1 (206) 825-4849
jyz2021@uw.edu

ABSTRACT

This project presents a computer vision-based hand gesture controlled smart home IoT system that enables intuitive, touchless control of multiple common household devices and provides an automatic emergency alert feature. A stationary camera connected to a Raspberry Pi 4 captures real-time video input, which is processed using OpenCV and Google MediaPipe to detect predefined hand gestures. Different gestures are mapped to control various devices, including a desk lamp and a fan motor, demonstrating practical smart home functionality in both illuminated and dark environments. To support operation in darkness, a custom-designed PCB integrates multiple LEDs and one photoresistor to provide adaptive lighting and environmental sensing for the vision system. When a designated distress gesture is detected, the system automatically triggers a phone call to a preset number through a paired mobile device via Bluetooth Low Energy (BLE) connection, alerting selected contacts without manual interaction. This gesture-based interface eliminates the need for physical switches or voice commands, making it particularly suitable for elderly users, individuals with mobility impairments, or situations requiring hands-free operation. The final experimental results show reliable gesture recognition and responsive device actuation across varying lighting conditions.

Keywords

Hand Gesture Recognition, Smart Home Automation, Internet of Things (IoT), Raspberry Pi, Computer Vision, MediaPipe, OpenCV, Emergency Alert System, Bluetooth Low Energy (BLE), Custom PCB Design

1. INTRODUCTION

Smart home technology has rapidly evolved in recent years as it enables users to interact with devices through mobile applications or voice assistants. While these interfaces offer convenience, they often require continuous phone access and clear speech input, which may not be accessible to elderly users, individuals with physical disabilities, or users in noisy or emergency situations. Specifically, traditional touch and voice-based interfaces can be inconvenient when hands are occupied or when silent interaction is preferred. These limitations motivate the exploration of alternative human-computer interactions for smart environments.

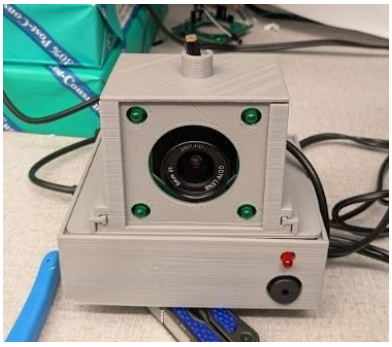

Figure 1. Demo device

Hand gesture recognition provides a natural, touchless, and speech-free interaction. The advances in computer vision and machine learning have made real-time, vision-based gesture recognition both feasible and affordable on embedded platforms. The integration of Internet of Things (IoT) technologies and low-computing devices such as the Raspberry Pi enables seamless control of physical hardware through software configurations. This project focuses on these developments to design and implement a gesture-driven control system with an integrated automatic emergency-call feature. By combining real-time video frame processing, BLE communication, and embedded hardware actuation, the system allows users to control appliances through simple hand gestures under both bright and low-light conditions. This paper describes the related work, system architecture, hardware and software implementation, final product evaluation, and future optimization for this system and other gesture-based assistive technologies.

2. RELATED WORK

Hand gesture recognition has been studied as an intuitive human computer interaction method for smart homes, assistive technology, and embedded IoT systems. Research over the past years has evolved from sensor-based systems toward vision-based and deep learning approaches that aim to provide reliable and real-time recognition using low-cost hardware. Early research in smart home gesture control relied heavily on specialized sensing devices, particularly cameras and structured light sensors. One representative study employed a Microsoft Kinect sensor combined with a Hidden Markov Model (HMM) to recognize dynamic hand gestures for controlling smart home appliances [1]. The system demonstrates that temporal probabilistic models can effectively capture continuous hand motion and translate it into device commands. However, such systems depend on expensive depth

sensing hardware that requires stable lighting and other conditions and are often limited to fixed indoor environments with calibrated sensor placement. These constraints reduce portability and limit adaptation to low cost and consumer ready settings. Our proposed system, on the other hand, relies solely on a standard RGB camera integrated with a Raspberry Pi. By using MediaPipe's pre-trained hand landmark detection model, the system avoids expensive depth sensors and custom training. This lowers our hardware costs and improves portability. Furthermore, the use of the camera combined with adaptive lighting through a custom PCB enables operation in both bright and low-light environments, which is a limitation of many earlier vision-only approaches.

More recent work has focused on applying machine learning and deep learning techniques to embedded devices for improved gesture recognition performance. A Raspberry Pi 5 based system utilized MediaPipe for landmark extraction and a custom feed-forward neural network trained on the HaGRIDv2 dataset to classify hand gestures in real time [2]. That system achieved high validation accuracy and smooth real-time operation at over twenty frames per second, demonstrating that lightweight neural networks can be effectively deployed on low-cost hardware. Similarly, a deep learning-based approach using a quantized convolutional neural network (CNN) with residual blocks achieved over 96% accuracy for American Sign Language digit recognition on Raspberry Pi platforms [3]. Model quantization and optimization techniques were applied to reduce memory and computation requirements, making deep learning feasible on non-GPU edge devices. The difference regarding recognition accuracy between this study and our system is that they require substantial dataset preparation, training, and optimization, which are the things we would plan to explore in the future. Our present project adopts MediaPipe's model and applies rule-based gesture interpretation using OpenCV. This design choice eliminates the need for large-scale data collection and neural network training. It still supports reliable real-time recognition. While we would consider training the hand tracking model by ourselves in the future, some of the tradeoff is that for practical smart home control devices involving a limited gesture set, the reduced complexity and faster development cycle offer significant advantages. And the rule-based systems may be less adaptable to highly complex or user-specific gestures compared to learned models.

Additionally, prior work has emphasized power efficiency and continuous sensing optimization. For example, one system integrated a motion sensor to disable vision processing when no one was present to reduce energy consumption. Similar to this concept, our system integrates photoresistors and light-sensing logic on a custom PCB, enabling adaptive LED illumination only when needed in the dark environment.

Beyond gesture-based control, smart home research has also explored security and intrusion detection. A research paper shows an integrated PIR motion sensor with a webcam and employed Histogram of Oriented Gradients (HOG) with a Support Vector Machine (SVM) for human detection [4]. When an intruder is detected, an alarm is triggered, and a notification message is sent to the user. This work demonstrated the feasibility of combining computer vision and IoT communication for safety use. This system focused on passive surveillance and human detection without too much user interaction while our system integrated active and user-initiated emergency signaling through a predefined alerting hand gesture. Instead of detecting intruders automatically, the system allows the user to trigger an emergency alert through a natural human gesture when needed.

Besides the safety monitoring products, there are other commercial smart home devices such as Amazon Alexa, Google Next, and Apple HomeKit that dominate the consumer market. These platforms rely primarily on voice commands and mobile applications as the main user interface. They offer robust cloud integration and mature mobile interfaces. However, they present several limitations in terms of accessibility and privacy. Voice-based interaction requires clear speech and quiet environments, which may not be available in emergency situations or for users with speech impairments. Gesture-based commercial solutions are limited and often rely on wearable sensors, radar systems, or some advanced cameras. Some smart gaming and VR systems incorporate gesture control, but these are normally not open source, nor easily adaptable for general-purpose smart home control. Our system, on the other hand, is fully local and low-cost. It doesn't depend on cloud services or proprietary hardware. All vision processing is performed directly on the Pi, which avoids user privacy issues and also ensures functionality without internet connection.

With respect to the need for efficient real-time solutions, recent work has also shifted toward on-device inference to reduce latency and improve privacy. An example is the On-Device Real-Time Hand Gesture Recognition system that integrates a hand skeleton tracker with a gesture classifier that runs entirely on local hardware [5]. It extends MediaPipe Hands by improving key point accuracy and adding world-metric 3D key point estimation for a more stable geometric reasoning. Their two complementary classifiers, one heuristic and one neural network, show that both rule-based and learned model can achieve high accuracy when paired with reliable landmark extraction. This work emphasizes the feasibility of achieving responsive gesture recognition using only a single RGB camera without requiring depth inputs or complex hardware.

When researched through modern patented technologies on gesture-based interaction systems, the trend of multimodal input was observed, such as both voice commands and biometric authentication. For example, recent patents describe systems that interpret hand movements, spoken instructions, and user-specific biometric patterns to control interfaces. These designs show flexible interaction styles that allow users to switch seamlessly between gestures and voice depending on environmental constraints. Multimodal also improves security as biometric signatures or voiceprints can be used to verify a user's identity for sensitive commands. At the same time, Apple has also patented methods for user personalization with customizable vision-based gesture interfaces that teach user-defined gestures through computer vision and machine learning. Rather than relying on a fixed set of pre-defined gestures, the system allows users to train custom gestures that fit their needs or preferences by extracting hand skeleton landmarks from camera imagery and applying adaptive machine learning models to classify both static and dynamic gestures.

In other research, some hybrid and alternative sensing approaches were also examined. For example, a comparative study by the authors of Advanced Technologies and Methods in Hand Gesture Analysis and Recognition Systems reviews both vision-based and non-vision-based methods, emphasizing how gesture recognition has evolved into multimodal and multialgorithm [6]. Their analysis spans data-glove sensors, traditional machine learning models,

such as HMMs, Naïve Bayes, color modeling), finite-state machine approaches for structured interaction, deep neural networks for high dimensional feature extraction. This study categorizes recognition into the canonical stages of detection, tracking, and recognition to compare how each algorithm addresses noise robustness, temporal variation, and inter-subject variability. Their results show that deep learning models outperform the classical methods in complex environments, even though they still face challenges related to dataset bias, occlusions, and deployment cost.

Other patent applications on sensor-based gestures combine infrared imaging, depth camera, and inertial measurement units (IMUs) together with wearable motion sensors to improve robustness in diverse environments. They are good for addressing challenges such as sensitivity to lighting conditions or background noise. Infrared and depth-based methods provide reliable performance in low light situations while wearable sensors reduce reliance on environmental factors by capturing body motion directly.

From what we have learned from this project, there can be multiple future research directions. One important extension involves continuous gesture recognition. Now our detection is primarily based on discrete frames. And many current systems also operate on distinct and segmented gestures. Incorporating temporal models such as recurrent neural networks, HMMs, or transformer-based structures could bring smoother interpretation of continuous hand motion and natural interaction. Another key research direction is the integration of personalized and adaptive gesture models like the patent from Apple mentioned above. User-specific calibration could improve robustness across different hand sizes, lighting conditions, and individual movement differences. Real-time adaptation without heavy computational need can be achieved by a hybrid system that combines MediaPipe's pre-trained landmark detection and lightweight on-device learning. Moreover, from a hardware perspective, future work could explore more of the use of infrared or depth cameras to reduce sensitivity to background noise. The custom PCB could also be extended to integrate more environmental sensors, such as temperature, humidity, or occupancy detection to support more comprehensive gesture interpretation based on context in the moment.

The emergency alert feature could be expanded through cloud services to enable multi-device alerts or remote monitoring. Although our current system uses local wireless communications with a mobile device, future implementations could explore secure cloud messaging protocols and check redundancy mechanisms to improve reliability. Finally, if we were to demonstrate the technical feasibility of the proposed system, we would need to conduct formal usability studies involving elderly and mobility and speech-impaired users. Besides that, security and privacy considerations, especially for in-home camera use, also need to be considered carefully with deeper investigation.

3. TECHINAL DETAILS

3.1 Overview

Our product controls an array of devices after receiving a visual command from the user. When a user displays a gesture from our predetermined gesture set, our product drives one or more of the devices in response. The product is able to control devices hard wired into it as well as via Bluetooth connection. This control involves turning devices on/off, switching them between modes, and even automatically sending phone calls. It is also able to function in the dark, with built-in LEDs that automatically turn on to assist the camera.

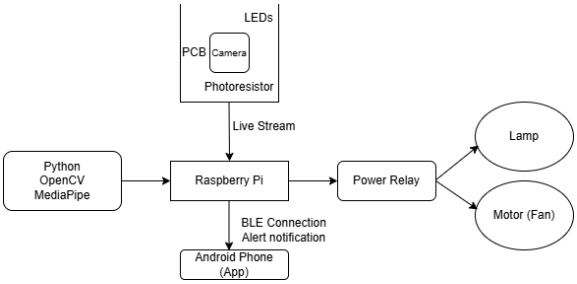

Figure 2. Block Diagram of the System

3.2 Theory of Operation

User input is captured using a camera and processed by a Raspberry Pi which uses MediaPipe and OpenCV to recognize the proper gestures. Once the gestures are identified they are mapped to output functions that the Pi executes when the gesture is received. The output functions that control hard wired devices are all handled by the Pi. They drive the GPIO pins to power the peripheral devices either directly on a breadboard or via an input to a relay. The output functions that handle the bluetooth devices live on both the Pi and the Android app downloaded on the device. The Pi sends a signal to the app to perform a task and the app itself handles the execution of those tasks.

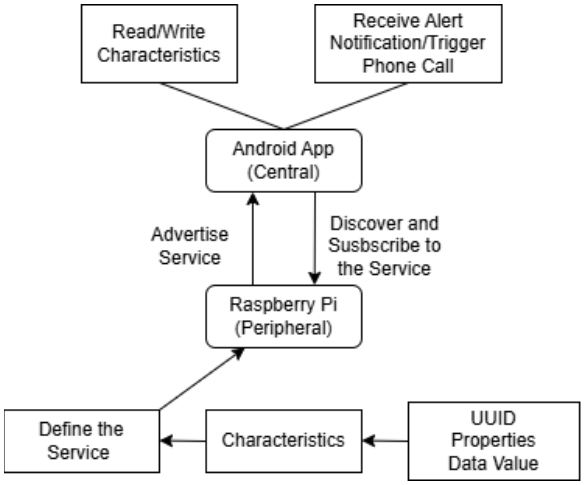

Figure 3. BLE Connection Architecture

3.3 Implementation Details

3.3.1 Hardware

Our product features both off the shelf as well as custom hardware. Off the shelf hardware includes a 12MP Raspberry Pi camera, a Raspberry Pi 4.0, and a 4-module relay board. Our custom hardware includes PCB and device housing.

The custom PCB implements two separate circuits: a light sensing circuit and an LED output circuit. The light sensing circuit uses a Light Dependent Resistor (LDR) to sense the brightness of the room. Due to the lack of analog input capability of the Raspberry Pi, we put it in parallel with a 100nF capacitor. The capacitor is first charged by a GPIO pin from the Pi, then another pin reads the logic level given by the capacitor as it's discharged. As the LDR changes resistance due to brightness, it changes the discharge time of the capacitor, with brighter environments resulting in shorter discharge, and darker environments resulting in longer discharge times. The time between turning off the input pin and the capacitor discharging long enough to give a logic level of 0 is measured by the Pi. When the time measured is above a certain amount, the Pi sends a high logic value to the LED output circuit to power the LEDs and brings it back to low when the time is lower than the set amount. In order to set the value that triggers the LED to toggle, we tested the LDR circuit in various lighting setups and read out the discharge time for each and found a value within the range between them so that it would trigger the LEDs to produce the desired outcome for our specific lighting conditions.

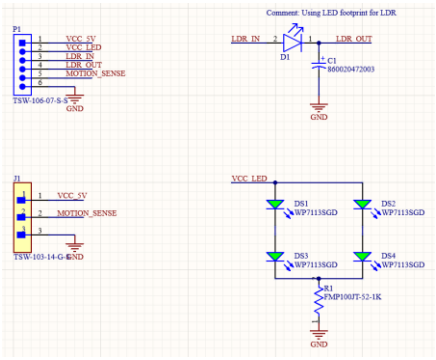

Figure 4. PCB Schematic

The board has a cutout for the camera lens to stick through, and the layout places the LEDs equidistant from each other around the cutout to allow for maximum lighting for the camera. All the signals going to and from the Pi are routed to one row of sockets.

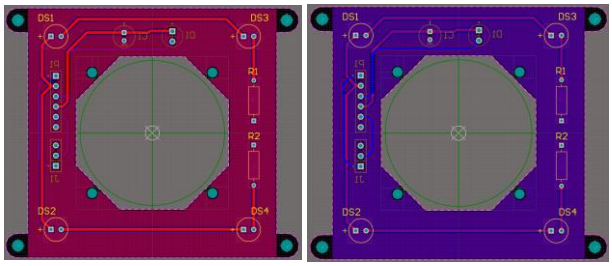

Figure 5. PCB Layout

The hard-wired peripheral devices sit on separate boards from the Pi and are controlled using the Pi's GPIO pins. The buzzer and emergency status LED sit on a breadboard and are directly powered by the Pi's output, meanwhile the lamp and fan are powered by a wall outlet and their connections to the outlet run through a relay that is driven by the Pi's logic output.

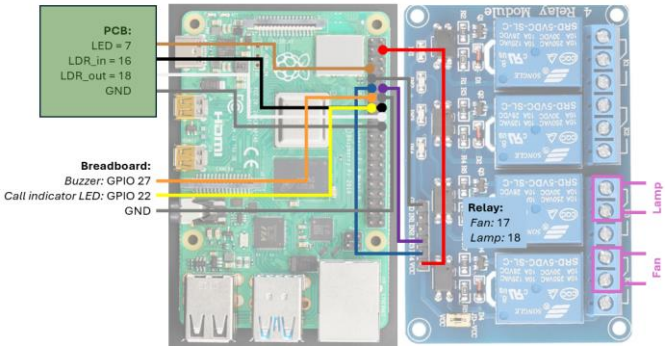

Figure 6. Device Pinouts

For the device housing, we implemented two 3D printed enclosures that could slot into one another. The top enclosure features two panels that go on the front of two devices; the front panel attaches to the front of the custom PCB, and the back panel splits in half and goes on either side of the camera PCB. The panels and their corresponding boards then slot into the top enclosure. The lid has a slot for the LDR to poke out of, so it faces away from the LEDs but still measures the room's brightness. The Raspberry Pi, relay board, and breadboard sit in the enclosure below, which has openings to connect to peripheral devices as well as display the buzzer and LED on the breadboard. The floor of the top enclosure and the lid of the bottom enclosure have openings to allow wires to pass between.

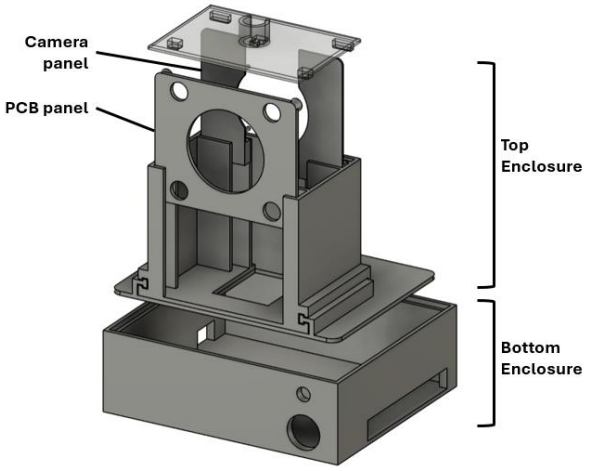

Figure 7. Device Housing

3.3.2 Software

Table 1. Gesture Functions

Gesture	Functions
Open hand	Lamp toggle
Closed fist	Fan toggle
Thumbs up	Single tone on buzzer
"I love you" sign	Plays scale on buzzer
Pointing up	Emergency call and flash indicator LED
Sliding index finger	Turn all devices off

The gesture recognition pipeline runs entirely on the Raspberry Pi and is implemented in Python using OpenCV and Google's MediaPipe for real-time hand landmark detection and gesture classification. A continuous video stream is captured from the Pi camera and converted into RGB frames, which are passed into the MediaPipe Gesture Recognizer configured in video mode with single-hand tracking. For each frame, the recognizer outputs both a set of 3D hand landmarks and a discrete gesture label (open_palm, closed_fist, thumbs_up, pointing_up, victory, or iloveyou). These gesture labels are then mapped to higher-level actions such as toggling GPIO pins, driving a buzzer, or triggering the emergency alert. The use of a pre-trained MediaPipe model eliminates the need for custom dataset collection and training while still providing accurate and low-latency recognition.

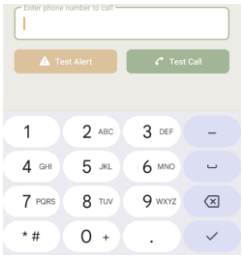

Figure 9. Number Fill-In-Slot to Place Call

The software implements a state machine on top of the raw gesture outputs. For the lamp and fan, corresponding to the open_palm and closed_fist gestures, the code maintains boolean toggle states that are inverted only when a gesture is newly observed after having been absent from the frame. This design prevents rapid flickering due to frame-to-frame classification jitter and ensures that performing open_palm → none → open_palm will reliably switch the lamp on and off. Additional logic prioritizes certain gestures over others for audio feedback. The victory gesture enables both output devices and drives the buzzer with a continuous tone. The thumbs_up gesture plays the same continuous tone, and the iloveyou gesture triggers a short C-major scale sequence using a PWM-controlled buzzer. All of these behaviors are implemented by mapping gesture labels to GPIO pin states and PWM frequency/duty-cycle settings in the main processing loop.

For the emergency feature, the pointing_up gesture is treated as a long-press action with temporal filtering to avoid accidental activation. When the system detects pointing_up, it starts a hold timer and requires the gesture to be maintained for at least three seconds before issuing an emergency alert. If the hold condition is satisfied and the cooldown period has expired, the Raspberry Pi's BLE service encodes a "CALL:phone_number" message into a GATT characteristic, which is then received by the companion Android application and translated into a phone call. At the same time, the software initiates a five-second LED blink sequence on a dedicated LED, implemented as a timer-driven toggle independent of the current gesture. Once this blink sequence is started, it runs to completion even if the user changes hand poses. The emergency logic also incorporates a cooldown timer to prevent repeated alerts from a single extended gesture.

In addition to static gestures, the system supports a dynamic motion gesture based on finger trajectory. The index fingertip (landmark 8) x-coordinate is tracked over a short temporal window, and a swipe gesture is detected when the fingertip moves consistently across the frame in one direction by more than a calibrated threshold. When this swipe is recognized, the software resets both the lamp and fan outputs to the off state and clears their internal toggle flags, effectively providing a "turn everything off" motion. The static gesture mapping, long-press timing, and swipe-based motion logic form a control layer that translates hand movements into smart home actions.

A custom Android mobile application was developed to serve as the BLE client for receiving emergency alerts from the Raspberry Pi and triggering user side actions. The application was built and tested using Android Studio and deployed onto a physical Android phone using USB debugging. Initial testing focused on verifying the notification system by triggering a local test alert using the button press to confirm correct notification permissions before BLE integration. To support BLE communication, runtime permissions were implemented to request access for Bluetooth scanning, connection, and find location services as required by Android security policies. The application layout was updated through XML to include interaction buttons and input fields, including a test call button and a phone number input interface as shown in Figure 9.

The BLE client functionality was implemented by initializing private fields for the Bluetooth adapter, BLE scanner, GATT connection, service UUID, and characteristic UUID in the MainActivity.java file. A ScanCallback was used to continuously search for nearby BLE peripherals. When the targeted device (the Pi) was discovered through service UUID matching, the application automatically initiated a GATT connection channel. Then upon successful connection, the application locates the predefined alert service and characteristic. The client then enabled notifications by subscribing to the Pi's NOTIFY characteristic through both setCharacteristicNotification() and by writing to the Client Characteristic Configuration Descriptor (CCCD), which allowed

the application to receive emergency alert messages asynchronously when a certain gesture was detected on the Pi.

During development, multiple permission-related bugs were identified and resolved. An initial design caused BLE scanning to fail because permissions were checked only after scanning had already started, which was fixed by validating permission before any scan operation. A second issue was because the phone call permission was included within the BLE permission checking function, which prevented service discovery. To resolve this, BLE permissions and phone call permissions were separated into two independent validation functions. The emergency call feature was implemented using Intent.ACTION_CALL, which directly placed the call through Google Voice using Wi-Fi calling, even without a physical SIM card installed. The system defaulted to Google Voice because it was configured as the device's primary calling application. When the Pi detects the alert gesture, its BLE script rends a notification to the mobile application in the format "CALL: phone number" through the NOTIFY characteristic. The app receives this message and parses the prefix to determine that a call should be placed.

For BLE testing before full system integration, nRF Connect was used as a simulated BLE peripheral. The same service and characteristics were configured on both the Android application and nRF Connect to enable device discovery and notification testing. Debug logging was added to the connectGatt() and onConnectionStateChange() functions to verify correct connection behavior after initial issue in automatic reconnection. The final system used a BLE script in Python on the Pi side to define the alert service, register alert characteristics, and advertise the device. During full integration, the Pi initially prompted Bluetooth pairing authorization. To eliminate this interruption and allow seamless automatic connection, the Pi Bluetooth agent was configured to accept pairing requests automatically, which enabled direct BLE connection without user interaction.

4. EVALUATION AND RESULTS

4.1 Computer vision

Table 2. Vision Hardware Configuration Performance

Vision Hardware	Average FPS	Result
Raspberry Pi Camera V@	~5	Model struggled to consistently track either hand of user.
Raspberry Pi HW Camera with 6mm 3MP Wide Angle Lens	~11	Model tracked the hand at a very reliable rate.

4.1.1 Raspberry Pi Camera V2

Initial performance benchmarking was conducted using the Raspberry Pi Camera V2. This configuration failed to meet the minimum design requirements, demonstrating a significant bottleneck in data acquisition. The average system output was limited to approximately 5 FPS, well below the target threshold. With our initial design, we were aiming to reach roughly 10-15 frames per second output from our end-to-end system. This metric is critical for ensuring the system maintains real-time processing capabilities, and the model performs with necessary accuracy.

4.1.2 Raspberry Pi HQ Camera with 6mm 3MP Wide Angle Lens

To address the latency and throughput issues, the imaging hardware was upgraded to a Raspberry Pi HQ Camera paired with a 6mm 3MP Wide Angle Lens. This hardware modification resulted in a measurable performance increase. The optimized configuration successfully doubled the system throughput, achieving a consistent average of around 10 FPS, effectively satisfying the lower bound of the project's performance goals.

4.2 Bluetooth

4.2.1 Initial Pairing Configuration

With our initial integration, the Raspberry Pi default security settings prompted the user for manual Bluetooth pairing authorization upon every connection attempt. This resulted in a system halt that required physical user interaction on the Pi side, failing our requirement for a seamless, headless operation.

4.2.2 Automated Agent Pairing Configuration

After we reconfigured the Pi Bluetooth agent to automatically accept pairing requests, we eliminated the need for manual approval. We achieved a seamless, automatic connection state where the Android client could discover, connect, and subscribe to the Pi without any user intervention.

5. DISCUSSION

5.1 Areas for Improvement

We found there were two main points for improvement: initial gesture recognition and peripheral device connection.

To start recognizing the gestures our model had to first recognize there's a hand in the frame and start tracking it, and this is where our product would behave inconsistently. When testing, we would have to move our open palms at different angles and positions until the camera locked onto it. Different lighting angles also affected the difficulty of achieving initial recognition, with back lighting being less effective than front or top lighting.

While it's a relatively minor issue, our other point for improvement is having our peripheral devices hard wired into our product. Having to cut open the cables for the devices and screw them into the device is not ideal for a customer experience, and it also limits where the devices can be placed since they are wired together and require proximity to an outlet

5.2 Future Improvements

To address our first point, we can improve initial gesture recognition by training our model in more diverse conditions. By using more lighting angles and hand positions during testing, we can increase the samples the model draws from. We can also adjust the camera LEDs to be more sensitive and trigger at a brighter level, so we can make sure there is a more consistent front lighting source.

For our second area of improvement, we can allow for a more seamless integration of peripheral devices by implementing two different changes. To remove the step of having to cut open the cables of the peripheral devices, we can have an outlet adapter with an internal relay, so the device could just plug into the adapter, and it would control the connection to the outlet. To allow for a farther

distance between the controlling device and peripherals, we could have the outlet adapters controlled via the Raspberry Pi's wireless LAN, eliminating the need to hardwire the internal relays to the Pi.

6. CONCLUSION

Overall, this product delivered proof of concept for our goal; we successfully used an array of gestures to control common household devices in various lighting conditions. Both our hardware and software solutions are comprised of custom designs as well as off-the-shelf solutions that we tuned for our use case. The path forward with this project would include covering more edge cases and optimizing the product for easier user experience.

7. REFERENCES

[1] Nigam, S., Shamoon, M., Dhasmana, S., & Choudhury, T. (2019). A complete study of methodology of hand gesture recognition system for smart homes. 2019 International Conference on Contemporary Computing and Informatics (IC3I), 289–294. https://doi.org/10.1109/IC3I46837.2019.9055608

[2] Hobbs, T., & Ali, A. (2025). Smart Home Control Using Real-Time Hand Gesture Recognition and Artificial Intelligence on Raspberry Pi 5. Electronics, 14(20), 3976. https://doi.org/10.3390/electronics14203976

[3] Yu, A., Qian, C., & Guo, Y. (2024). A real-time hand gesture recognition system on Raspberry Pi: A deep learning-based approach. 2024 IEEE 21st Consumer Communications & Networking Conference (CCNC), 499–506. https://doi.org/10.1109/CCNC51664.2024.10454652

[4] Surantha, Nico, and Wingky R. Wicaksono. "An IoT based house intruder detection and alert system using histogram of oriented gradients." Journal of Computer Science 15.8 (2019): 1108-1122.

[5] Sung, G., Sokal, K., Uboweja, E., Bazarevsky, V., Baccash, J., Bazavan, E. G., Chang, C.-L., & Grundmann, M. (2021). On-device real-time hand gesture recognition (arXiv:2111.00038). arXiv. https://arxiv.org/abs/2111.00038

[6] Rahman, M. M., Uzzaman, A., Khatun, F., Aktaruzzaman, M., & Siddique, N. (2025). A comparative study of advanced technologies and methods in hand gesture analysis and recognition systems. Expert Systems with Applications, 266, 125929. https://doi.org/10.1016/j.eswa.2024.125929